

**UNLaM**Dto. Ingeniería e Investigaciones Tecnológicas

Objetivo: Comprender el concepto de Recursividad, sus características y uso para la resolución de problemas en informática.

1. Introducción a la Recursividad

Definición de Recursividad

La recursividad es una técnica de programación en la que una función se llama a sí misma directamente o indirectamente para resolver una porción de un problema más grande mediante la solución de casos más pequeños y similares. Es un enfoque muy utilizado en programación debido a su capacidad para simplificar problemas complejos que pueden ser divididos en sub problemas más simples y manejables.

Propósito de la Recursividad

El principal propósito de la recursividad es dividir un problema complejo en sub problemas más simples de la misma naturaleza, facilitando su solución. Al utilizar la recursividad, un programador puede escribir menos código para resolver problemas que de otra manera requerirían estructuras iterativas complejas. La recursividad es especialmente útil en casos donde el problema en sí mismo es definido recursivamente, como es el caso de ciertos cálculos matemáticos, operaciones en estructuras de datos recursivas o algoritmos que requieren exploración de numerosos casos o configuraciones, como los algoritmos de búsqueda y ordenamiento.

Ventajas de usar la Recursividad

- **Claridad y reducción de código:** A menudo, el código recursivo es más fácil de leer y más corto que su equivalente iterativo.
- **Solución natural para problemas recursivos:** Algunos problemas son inherentemente recursivos (como caminar por estructuras de datos recursivas o descomponer problemas en subproblemas idénticos). Para estos problemas, la recursividad es la solución más natural y directa.



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

- **Uso de memoria automático:** La recursividad se beneficia del manejo automático de la pila de llamadas del lenguaje de programación, almacenando el estado de cada llamada y sus variables locales.

Desventajas de la Recursividad

- **Costo en rendimiento:** Cada llamada recursiva añade una nueva capa a la pila de llamadas, lo que puede llevar a un consumo significativo de memoria y, eventualmente, a un desbordamiento de pila si la recursividad es demasiado profunda.
- **Eficiencia:** En algunos casos, la recursividad puede ser menos eficiente que las soluciones iterativas, especialmente si se realiza un gran número de llamadas redundantes que podrían ser evitadas en un enfoque iterativo.

Ejemplos de Problemas que se Benefician de la Recursividad

- **Cálculos matemáticos** como la serie de Fibonacci, factorial de un número, entre otros, presentan definiciones recursivas naturales.
- **Algoritmos de división y conquista**, como el ordenamiento rápido (quicksort) y la búsqueda binaria, donde el problema se divide en subproblemas más pequeños que son más fáciles de manejar.
- **Operaciones en estructuras de datos recursivas**, como la búsqueda o inserción en árboles binarios, donde cada operación en una estructura puede ser expresada en términos de operaciones en sus subestructuras.

2. Concepto de Recursividad

Recursividad vs. Iteración

La recursividad y la iteración son dos métodos fundamentales para repetir operaciones en programación. Ambos métodos pueden ser utilizados para lograr resultados similares, pero operan de maneras muy diferentes:



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

- **Recursividad:** Una función recursiva es aquella que se llama a sí misma para resolver iterativamente sub problemas más pequeños que son versiones reducidas del problema original. La recursividad es a menudo vista como una aproximación de "divide y vencerás". Cada llamada a la función crea una nueva capa en la pila de llamadas, lo que permite que el estado de cada instancia de problema sea guardado automáticamente.
- **Iteración:** Utiliza estructuras de control de bucle como **for** o **while** para repetir una serie de operaciones. La iteración es generalmente más eficiente en términos de uso de memoria, ya que no requiere la sobrecarga adicional de múltiples llamadas a funciones y gestión de la pila de llamadas.

Comparación:

- **Claridad:** La recursividad puede simplificar el código y hacerlo más fácil de entender, especialmente cuando el problema en sí es inherente o naturalmente recursivo.
- **Rendimiento:** Los bucles iterativos suelen ser más eficientes desde el punto de vista del rendimiento, ya que no involucran el coste adicional de llamadas a funciones y uso de la pila de llamadas.
- **Riesgo de errores:** La recursividad incorrectamente implementada puede llevar a desbordamientos de pila y errores de lógica. La iteración puede resultar en bucles infinitos si no se maneja correctamente la condición de terminación.

Funciones Recursivas

Una función recursiva es esencialmente una función que se llama a sí misma durante su propia ejecución. Para que una función sea efectivamente recursiva, debe tener las siguientes características:

- **Caso base:** Cada función recursiva debe tener una condición de terminación, conocida como "caso base", para evitar la recursión infinita y eventualmente resolver el problema.



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

El caso base define el punto en el cual la recursión debe detenerse porque el problema se ha reducido a su forma más simple.

- **Regla de recursión o Caso recursivo:** Debe haber reglas que reduzcan progresivamente cada problema hacia el caso base. Esto generalmente involucra la función llamándose a sí misma con un conjunto diferente de argumentos que acercan el problema al caso base.

3. Elementos característicos de las funciones recursivas

Para que una función recursiva funcione correctamente y de manera eficiente, debe incorporar algunos elementos esenciales:

Llamada a sí misma

Uno de los aspectos definitorios de una función recursiva es su capacidad para llamarse a sí misma durante su ejecución. Esta auto-referencia permite que la función aborde una versión más pequeña del problema inicial en cada llamada sucesiva.

- **Estructura de una llamada recursiva:**
 - La función primero verifica si el caso actual puede ser resuelto sin más recursión (esto usualmente involucra el caso base).
 - Si no es el caso base, la función se llama a sí misma con nuevos parámetros que acercan el problema hacia una solución.

Condición de corte o caso base

El caso base es crucial en cualquier función recursiva. Es la condición bajo la cual la función deja de llamarse a sí misma y comienza a devolver valores. Sin un caso base adecuado, una función recursiva podría continuar ejecutándose indefinidamente, llevando a un desbordamiento de pila y a un eventual colapso del programa.



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

- **Importancia del caso base:**

- Previene la recursión infinita asegurando que cada llamada recursiva progrese hacia el caso base.
- Define el resultado más simple del problema, permitiendo que las llamadas recursivas anteriores se resuelvan.

Reglas de reducción

Para que la recursión sea efectiva, cada llamada recursiva debe simplificar el problema. Esto generalmente se logra alterando los argumentos de la función de manera que cada nueva llamada se acerque al caso base.

Efecto del entorno de llamada

Cada llamada a una función recursiva tiene su propio entorno de ejecución en la pila de llamadas. Esto incluye sus propias copias de las variables locales y los parámetros, lo que permite que cada ejecución de la función sea independiente de las demás.

- **Manejo de la pila de llamadas:**

- Cada llamada a una función recursiva agrega una nueva "capa" a la pila de llamadas, que se almacena en la memoria.
- Cuando una llamada alcanza el caso base, comienza a devolver valores y "desapila" la pila, resolviendo las llamadas pendientes en orden inverso a cómo fueron llamadas.

4. Ejemplo de aplicación

Vamos a analizar la implementación recursiva de la función **factorial** de un número. Si bien el ejemplo puede resultar sencillo, nos va a permitir enfocarnos en el concepto de recursividad sin perdernos en el problema en sí, que es calcular el factorial de un número entero.



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

En la Figura 1 podemos ver como se puede descomponer la función Factorial en su **caso base** y su **caso recursivo**. En general, cuando se trata de algoritmos recursivos, esta fase del análisis es crucial debido a que en ello se basa el diseño e implementación.

$$\text{fact}(n) = \begin{cases} \text{si } n = 0 & \longrightarrow 1 \\ \text{si } n > 0 & \longrightarrow n \cdot \text{fact}(n - 1) \end{cases}$$

Figura 1. Descomposición de la función factorial en su caso base y recursivo.

De nuevo en la Figura 1, podemos extraer:

1. **Caso Base:** El factorial de 0 es 1 ($0! = 1$).
2. **Caso Recursivo:** El factorial de cualquier número mayor a cero es $n \cdot \text{factorial}(n-1)$. En este paso se puede ver como el problema se reduce progresivamente (en cada llamada n se reduce en 1) hacia el caso base, este es el ejemplo de la **regla de reducción**.

Analicemos paso a paso lo que sucede con las llamadas recursivas (**Fase 1**) en el cálculo del factorial de 4 (**factorial(4)**):

1. **Llamada inicial: factorial(4)**
 - La función verifica si $n == 0$. No lo es, así que procede a la siguiente línea.
 - Retorna $4 * \text{factorial}(3)$. Necesita el resultado de **factorial(3)** para continuar.
2. **Siguiente llamada: factorial(3)**
 - Similar a la llamada anterior, verifica la condición base y luego retorna $3 * \text{factorial}(2)$.
3. **Siguiente llamada: factorial(2)**
 - Procede como antes, retornando $2 * \text{factorial}(1)$.
4. **Siguiente llamada: factorial(1)**
 - Retorna $1 * \text{factorial}(0)$, ya que aún no es la condición base.



5. Condición base alcanzada: **factorial(0)**

- Ahora **n == 0**, por lo que la función retorna **1**

Como se puede ver, en los pasos 1 a 5 no se realizó cálculo alguno. Lo que ha sucedido es que se han realizado múltiples llamadas a la función factorial hasta que es alcanzado el **caso base** (recordemos que en nuestro ejemplo es **n = 0**).

Ahora es el momento de la resolución de las llamadas recursivas (**Fase 2**):

Una vez que **factorial(0)** retorna **1**, cada llamada anterior puede ahora completar su cálculo:

- **factorial(1)** recibe el resultado de **1** y lo multiplica por **1**, resultando en **1**.
- **factorial(2)** recibe **1** de **factorial(1)** y lo multiplica por **2**, resultando en **2**.
- **factorial(3)** recibe **2** de **factorial(2)** y lo multiplica por **3**, resultando en **6**.
- **factorial(4)** recibe **6** de **factorial(3)** y lo multiplica por **4**, resultando en **24**.

Por lo tanto, el resultado final de **factorial(4)** es **24**.

Este proceso ilustra cómo las llamadas recursivas se apilan y luego se resuelven en orden inverso, devolviendo el control y el resultado a la llamada anterior hasta que la primera llamada completa el cálculo final.

En la Figura 2 se puede ver la implementación de la función. En la línea 4 comienza el desarrollo de la función factorial. La línea 6 verifica si se ha alcanzado el **caso base**. En el caso de que no se haya alcanzado, en la línea 9 se hace la llamada recursiva. En ese caso la función es llamada con el valor **n - 1** (es importante recordar que **n - 1** es distinto de **n = n - 1**) como argumento, lo que implica que el problema se va reduciendo hacia el caso base.



```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int factorial(int n)
5  {
6      if(n == 0)
7          return 1;
8      else
9          return n * factorial(n - 1);
10 }
11
12 int main()
13 {
14     int n = 4;
15     int resultado;
16
17     resultado = factorial(n);
18     printf("El factorial de %d es %d\n", n, resultado);
19     return 0;
20 }
```

Figura 2. Implementación recursiva de la función factorial de un número entero.

Para ayudar a la explicación, vamos a modificar levemente la función como se muestra en la Figura 3, agregando un par de variables (*fact* y *retorno*) para almacenar temporalmente los valores y mejorar la comprensión de lo que está sucediendo. Estas dos variables son únicamente utilizadas con fines didácticos.

En la Figura 4 podemos observar en modo **Debug** las condiciones iniciales del programa. La ejecución se encuentra detenida justo antes de la llamada a la función **factorial**. Para poder observar las llamadas es necesario que se encuentre activada la ventana **Call Stack** y para poder observar la evolución de las variables debe encontrarse activa la ventana **Watches**.



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int factorial(int n)
5  {
6      int retorno = 0;
7      int fact = 0;
8
9      if (n == 0)
10         return 1;
11     else
12     {
13         /* return n * factorial(n - 1); */
14         fact = factorial(n - 1);
15         retorno = n * fact;
16         return retorno;
17     }
18 }
19
20 int main()
21 {
22     int n = 4;
23     int resultado;
24
25     resultado = factorial(n);
26     printf("El factorial de %d es %d\n", n, resultado);
27     return 0;
28 }
```

Figura 3. Implementación recursiva de la función factorial con variables temporales para visualizar los resultados intermedios.



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

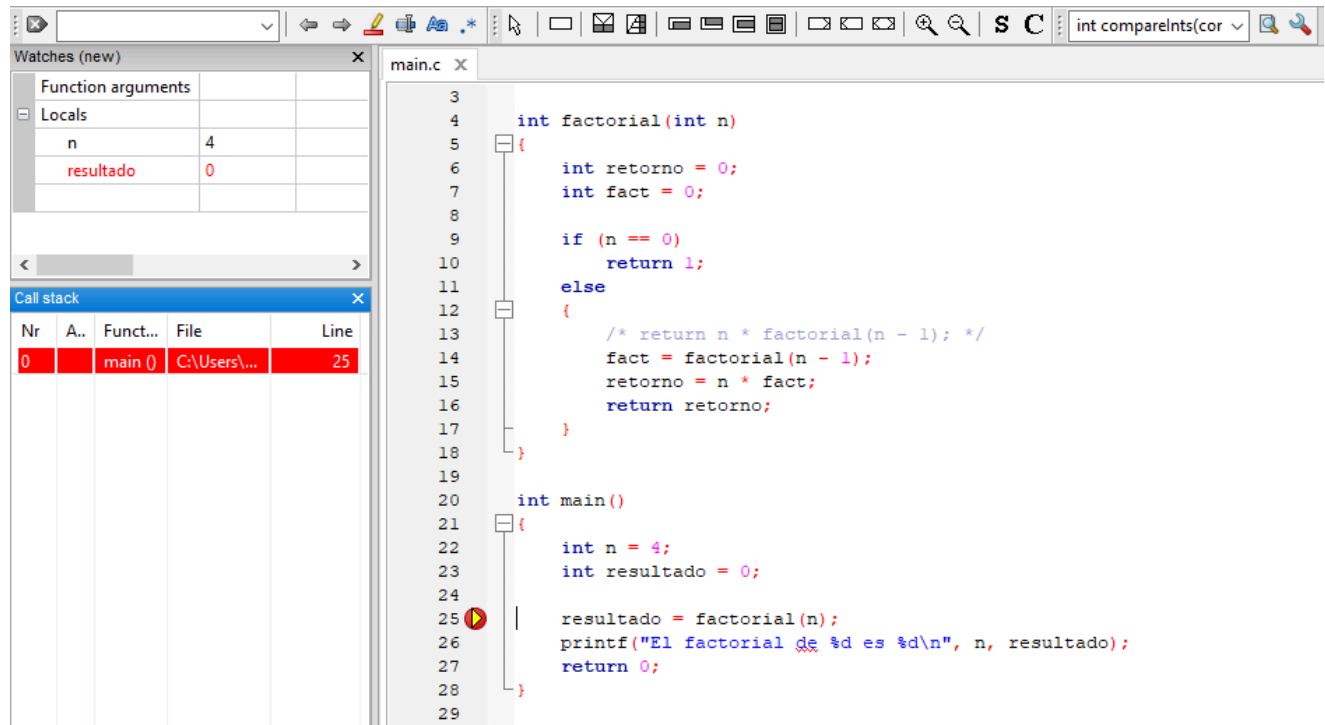


Figura 4. Vista en modo Debug de las condiciones iniciales del programa.

Analicemos lo que sucede durante la **Fase 1** con las llamadas recursivas. Recordemos que en una llamada recursiva se hacen sucesivas llamadas a la función hasta alcanzar la el caso base. Vayamos a esta fase. Observar el valor de la variable **n**, y la pila de llamadas (**Call stack**):



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

1. Llamada inicial: factorial(4)

Watches (new)

Function arguments	
n	4

Locals

retorno	0
fact	0

Call stack

Nr	Address	Function
0	0x401583	factorial (n=4)
1	0x4015be	main()

main.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int factorial(int n)
5  {
6      int retorno = 0;
7      int fact = 0;
8
9      if (n == 0)
10         return 1;
11     else
12     {
13         /* return n * factorial(n - 1); */
14         fact = factorial(n - 1);
15         retorno = n * fact;
16         return retorno;
17     }
18 }
19
20 int main()
21 {
22     int n = 4;
23     int resultado = 0;
24
25     resultado = factorial(n);
26     printf("El factorial de %d es %d\n", n, resultado);
```

2. Siguiendo llamada: factorial(3)

Watches (new)

Function arguments	
n	3

Locals

retorno	0
fact	0

Call stack

Nr	Address	Function
0	0x401583	factorial (n=3)
1	0x401583	factorial (n=4)
2	0x4015be	main()

main.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int factorial(int n)
5  {
6      int retorno = 0;
7      int fact = 0;
8
9      if (n == 0)
10         return 1;
11     else
12     {
13         /* return n * factorial(n - 1); */
14         fact = factorial(n - 1);
15         retorno = n * fact;
16         return retorno;
17     }
18 }
19
20 int main()
21 {
22     int n = 4;
23     int resultado = 0;
24
25     resultado = factorial(n);
26     printf("El factorial de %d es %d\n", n, resultado);
```



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

3. Siguiendo llamada: factorial(2)

Watches (new)

Function arguments	
n	2

Locals

retorno	0
fact	0

Call stack

Nr	Address	Function
0		factorial (n=2)
1	0x401583	factorial(n=3)
2	0x401583	factorial(n=4)
3	0x4015be	main()

main.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int factorial(int n)
5  {
6      int retorno = 0;
7      int fact = 0;
8
9      if (n == 0)
10         return 1;
11     else
12     {
13         /* return n * factorial(n - 1); */
14         fact = factorial(n - 1);
15         retorno = n * fact;
16         return retorno;
17     }
18 }
19
20 int main()
21 {
22     int n = 4;
23     int resultado = 0;
24
25     resultado = factorial(n);
26     printf("El factorial de %d es %d\n", n, resultado);

```

4. Siguiendo llamada: factorial(1)

Watches (new)

Function arguments	
n	1

Locals

retorno	0
fact	0

Call stack

Nr	Address	Function
0		factorial (n=1)
1	0x401583	factorial(n=2)
2	0x401583	factorial(n=3)
3	0x401583	factorial(n=4)
4	0x4015be	main()

main.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int factorial(int n)
5  {
6      int retorno = 0;
7      int fact = 0;
8
9      if (n == 0)
10         return 1;
11     else
12     {
13         /* return n * factorial(n - 1); */
14         fact = factorial(n - 1);
15         retorno = n * fact;
16         return retorno;
17     }
18 }
19
20 int main()
21 {
22     int n = 4;
23     int resultado = 0;
24
25     resultado = factorial(n);
26     printf("El factorial de %d es %d\n", n, resultado);

```



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

5. Condición base alcanzada: factorial(0)

The screenshot shows a debugger window with three main panels:

- Watches (new):**
 - Function arguments: n = 0
 - Locals: retorno = 0, fact = 0
- Call stack:**

Nr	Address	Function
0		factorial(n=0)
1	0x401583	factorial(n=1)
2	0x401583	factorial(n=2)
3	0x401583	factorial(n=3)
4	0x401583	factorial(n=4)
5	0x4015be	main()
- main.c:**

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int factorial(int n)
5  {
6      int retorno = 0;
7      int fact = 0;
8
9      if (n == 0)
10         return 1;
11     else
12     {
13         /* return n * factorial(n - 1); */
14         fact = factorial(n - 1);
15         retorno = n * fact;
16         return retorno;
17     }
18 }
19
20 int main()
21 {
22     int n = 4;
23     int resultado = 0;
24
25     resultado = factorial(n);
26     printf("El factorial de %d es %d\n", n, resultado);

```

En este punto “***n***” toma el valor cero, y por este motivo se alcanza la condición base y comienza la **Fase 2** con la resolución de las llamadas recursivas. En esta etapa, cada llamada apilada en el stack será desapilada y retornará el valor calculado por la función. En este caso el valor de retorno será 1 (línea 10) correspondiente a **factorial(0)**. Para todos los demás casos, el valor a retornar será almacenado en la variable **retorno** para observar lo que sucede.



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

- **factorial(1)** recibe el resultado de **1** y lo multiplica por **1**, resultando en **1**.

Watches (new) X

Function arguments	
n	1

Locals

retorno	1
fact	1

Call stack X

Nr	Address	Function
0		factorial (n=1)
1	0x401583	factorial(n=2)
2	0x401583	factorial(n=3)
3	0x401583	factorial(n=4)
4	0x4015be	main()

main.c X

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int factorial(int n)
5  {
6      int retorno = 0;
7      int fact = 0;
8
9      if (n == 0)
10         return 1;
11     else
12     {
13         /* return n * factorial(n - 1); */
14         fact = factorial(n - 1);
15         retorno = n * fact;
16         return retorno;
17     }
18 }
19
20 int main()
21 {
22     int n = 4;
23     int resultado = 0;
24
25     resultado = factorial(n);
26     printf("El factorial de %d es %d\n", n, resultado);

```

- **factorial(2)** recibe **1** de **factorial(1)** y lo multiplica por **2**, resultando en **2**.

Watches (new) X

Function arguments	
n	2

Locals

retorno	2
fact	1

Call stack X

Nr	Address	Function
0		factorial (n=2)
1	0x401583	factorial(n=3)
2	0x401583	factorial(n=4)
3	0x4015be	main()

main.c X

```

4  int factorial(int n)
5  {
6      int retorno = 0;
7      int fact = 0;
8
9      if (n == 0)
10         return 1;
11     else
12     {
13         /* return n * factorial(n - 1); */
14         fact = factorial(n - 1);
15         retorno = n * fact;
16         return retorno;
17     }
18 }
19
20 int main()
21 {
22     int n = 4;
23     int resultado = 0;
24
25     resultado = factorial(n);
26     printf("El factorial de %d es %d\n", n, resultado);
27     return 0;
28 }

```



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

- **factorial(3)** recibe 2 de **factorial(2)** y lo multiplica por 3, resultando en 6.

Watches (new)

Function arguments	
n	3

Locals

retorno	6
fact	2

Call stack

Nr	Address	Function
0		factorial (n=3)
1	0x401583	factorial(n=4)
2	0x4015be	main()

```

4  int factorial(int n)
5  {
6      int retorno = 0;
7      int fact = 0;
8
9      if (n == 0)
10         return 1;
11     else
12     {
13         /* return n * factorial(n - 1); */
14         fact = factorial(n - 1);
15         retorno = n * fact;
16         return retorno;
17     }
18 }
19
20 int main()
21 {
22     int n = 4;
23     int resultado = 0;
24
25     resultado = factorial(n);
26     printf("El factorial de %d es %d\n", n, resultado);
27     return 0;
28 }
  
```

- **factorial(4)** recibe 6 de **factorial(3)** y lo multiplica por 4, resultando en 24.

Watches (new)

Function arguments	
n	4

Locals

retorno	24
fact	6

Call stack

Nr	Address	Function
0		factorial (n=4)
1	0x4015be	main()

```

4  int factorial(int n)
5  {
6      int retorno = 0;
7      int fact = 0;
8
9      if (n == 0)
10         return 1;
11     else
12     {
13         /* return n * factorial(n - 1); */
14         fact = factorial(n - 1);
15         retorno = n * fact;
16         return retorno;
17     }
18 }
19
20 int main()
21 {
22     int n = 4;
23     int resultado = 0;
24
25     resultado = factorial(n);
26     printf("El factorial de %d es %d\n", n, resultado);
27     return 0;
28 }
29
  
```



Por lo tanto, el resultado final de **factorial(4)** es **24**.

```
Watches (new) X
Function arguments
Locals
n 4
resultado 24

Call stack X
Nr A.. Function
0 main()

main.c X
4 int factorial(int n)
5 {
6     int retorno = 0;
7     int fact = 0;
8
9     if (n == 0)
10        return 1;
11    else
12    {
13        /* return n * factorial(n - 1); */
14        fact = factorial(n - 1);
15        retorno = n * fact;
16        return retorno;
17    }
18 }
19
20 int main()
21 {
22     int n = 4;
23     int resultado = 0;
24
25     resultado = factorial(n);
26     printf("El factorial de %d es %d\n", n, resultado);
27     return 0;
28 }
```

5. Tipos de problemas resueltos mediante recursión

Como dijimos anteriormente, la recursividad es particularmente útil para abordar ciertos tipos de problemas en programación. Estos problemas suelen compartir características que los hacen ideales para ser abordados con soluciones recursivas. Veamos algunos de estos problemas.

Problemas divisibles

La recursividad se ajusta bien para resolver problemas que pueden ser divididos en problemas más pequeños, cada uno de los cuales es una versión más pequeña del problema original. Estos problemas se suelen resolver de manera más eficiente con un enfoque de "divide y vencerás".

- **Ejemplos comunes:**

- **Ordenamiento y búsqueda:** Algoritmos como quicksort y mergesort (estos algoritmos se estudian en materias posteriores) dividen el conjunto de datos en partes más pequeñas, ordenan esas partes y luego las combinan. De manera



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

similar, la búsqueda binaria divide repetidamente un conjunto ordenado para buscar un elemento.

- **Cálculo de expresiones matemáticas:** Problemas como calcular potencias, el máximo común divisor, y otras operaciones matemáticas pueden descomponerse en operaciones más simples sobre números más pequeños.

Problemas basados en elecciones consecutivas

Algunos problemas requieren explorar varios caminos o decisiones, donde cada decisión lleva a más decisiones. La recursividad puede simplificar la lógica necesaria para explorar estas múltiples vías.

- **Ejemplos comunes:**
 - **Permutaciones y combinaciones:** Generar todas las posibles formas de organizar o seleccionar elementos.
 - **Problemas de backtracking:** Como el problema de las N reinas o el laberinto, donde se debe probar una serie de decisiones y retroceder si no conducen a una solución.

Problemas que usan subestructuras óptimas

Los problemas que se pueden resolver optimizando soluciones a sub problemas más pequeños y luego combinando estos resultados para obtener la solución al problema más grande se benefician enormemente de la recursividad. Esto es típico en técnicas de programación dinámica.

- **Ejemplos comunes:**
 - **Corte de varillas:** Maximizar el beneficio al cortar y vender piezas de una varilla de longitud dada.
 - **Distancia de edición (Levenshtein):** Determinar cuántas ediciones (inserciones, eliminaciones, sustituciones) se necesitan para convertir una cadena de texto en otra.



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

Problemas con estructuras de datos recursivas (se estudiarán en materias sucesivas)

La recursividad es naturalmente adecuada para trabajar con estructuras de datos que se definen a sí mismas recursivamente, como los árboles y las listas enlazadas que serán abordadas en próximas asignaturas.

- **Ejemplos comunes:**

- **Árboles binarios:** Operaciones como búsqueda, inserción y eliminación en árboles binarios se realizan recursivamente, tratando cada subárbol como un árbol en sí mismo.
- **Recorrido de grafos:** Técnicas como DFS (búsqueda en profundidad) que pueden implementarse recursivamente para explorar todos los nodos de un grafo.

En resumen, estos tipos de problemas son especialmente adecuados para tratarse mediante recursividad porque permiten descomponer el problema en instancias más pequeñas o manejar estructuras de datos de manera que cada parte pueda ser tratada como un problema menor de la misma índole. La recursividad proporciona una manera clara y elegante de abordar estos problemas, aunque es importante tener en cuenta la **eficiencia** y el **uso de recursos**, especialmente con respecto al *tamaño de la pila de llamadas* y el *riesgo de desbordamiento de pila*.

6. Ventajas y Desventajas de Implementaciones Recursivas vs. Iterativas

La elección entre recursividad e iteración depende en gran medida del problema específico, las limitaciones de recursos, la claridad del código, y la experiencia y preferencia del programador. Veamos sus respectivas ventajas y desventajas de cara a poder realizar elecciones informadas al enfrentarse a problemas de programación reales.

Ventajas de la Recursividad

1. **Claridad y simplicidad:**

**UNLaM**Dto. Ingeniería e Investigaciones Tecnológicas

- La recursividad puede hacer que el código sea más claro y fácil de entender, especialmente cuando el problema en sí es recursivo por naturaleza, como la definición de árboles binarios o el cálculo de los números de Fibonacci.
- Las soluciones recursivas a menudo requieren menos líneas de código, lo que puede reducir la complejidad y mejorar la legibilidad.

2. Reducción de la necesidad de estructuras auxiliares:

- En muchos casos, las soluciones recursivas no requieren la creación explícita de estructuras de datos auxiliares, como pilas o colas, que son necesarias en implementaciones iterativas para problemas como el recorrido de árboles o grafos.

3. Aprovechamiento de la pila de llamadas:

- La recursividad utiliza la pila de llamadas del sistema para mantener el estado de cada llamada recursiva, facilitando la gestión de estados sin necesidad de implementación adicional.

Desventajas de la Recursividad**1. Costo de memoria y de rendimiento:**

- Cada llamada recursiva añade una nueva capa a la pila de llamadas, lo cual consume memoria. Esto puede llevar a un desbordamiento de pila en casos extremos donde la profundidad de la recursividad es muy alta.
- La recursividad puede ser menos eficiente en términos de tiempo de ejecución debido a la sobrecarga de las múltiples llamadas a funciones y el manejo de la pila.

2. Complejidad de depuración:

**UNLaM**Dto. Ingeniería e Investigaciones Tecnológicas

- Las funciones recursivas pueden ser más difíciles de depurar debido a su naturaleza auto-referencial y al manejo del estado a través de la pila de llamadas.

Ventajas de la Iteración**1. Eficiencia de memoria:**

- Las iteraciones no utilizan la pila de llamadas de la misma manera que las funciones recursivas, lo que generalmente resulta en un uso más eficiente de la memoria.

2. Mejor rendimiento en tiempo de ejecución:

- Las soluciones iterativas a menudo ejecutan más rápido que las recursivas debido a la ausencia de la sobrecarga involucrada en las múltiples llamadas a funciones y la gestión de la pila.

3. Facilidad de depuración:

- Las soluciones iterativas suelen ser más fáciles de seguir y depurar, ya que el flujo de control es lineal y no implica llamadas complejas ni gestión de pila.

Desventajas de la Iteración**1. Complejidad de código en problemas recursivos naturales:**

- Para problemas inherentemente recursivos, las soluciones iterativas pueden ser más complejas y menos intuitivas, lo que requiere estructuras adicionales y una lógica más complicada para manejar estados.

2. Necesidad de estructuras de datos adicionales:

- Algunos algoritmos, como los recorridos de profundidad en grafos o árboles, pueden requerir explícitamente estructuras de datos como pilas para implementar versiones iterativas que emulan el comportamiento de una solución recursiva.

**UNLaM**Dto. Ingeniería e Investigaciones Tecnológicas

7. Recursividad y Memoria

Entender cómo la recursividad interactúa con la memoria de la computadora es esencial para diseñar programas eficientes y que se ajusten al requerimiento del problema a resolver. Sin embargo, es crucial prevenir un error bastante común como el desbordamiento de la pila (especialmente en sistemas con recursos limitados como los embebidos). En esta sección vamos a tratar algunos de los puntos fundamentales del manejo de la memoria durante las llamadas recursivas.

Pila de Llamadas

La pila de llamadas, o "call stack", es una estructura de datos que utiliza la mayoría de los sistemas operativos y lenguajes de programación para manejar la ejecución de subrutinas o funciones durante un programa. En el contexto de la recursividad, la pila de llamadas juega un papel crucial:

- **Funcionamiento de la Pila:**

- Cada vez que una función es llamada, un "marco o entorno de pila" (o "stack frame") se crea y se añade a la pila. Este marco incluye la dirección de retorno (el punto del programa donde debe continuar la ejecución después de que la función retorne), los argumentos de la función, las variables locales y el estado actual del registro.
- Cuando una función retorna, su entorno de pila correspondiente es eliminado de la pila, y el control del programa se transfiere de nuevo al punto de retorno guardado en el entorno.

- **Recursividad en la Pila:**

- En las llamadas recursivas, cada llamada a la función crea un nuevo entorno en la pila. Si la recursividad es muy profunda, esto puede resultar en una gran cantidad de entornos almacenados en la pila, lo que potencialmente podría llevar



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

a un desbordamiento de la pila, donde se excede la capacidad de memoria reservada para la pila.

Visualización de la Pila de Llamadas

Para entender mejor la recursividad, puede ser útil visualizar cómo la pila de llamadas se modifica con cada llamada y retorno de función. Esto se puede realizar con las herramientas provistas por el IDE de desarrollo. Recuerden que en ejemplo del cálculo del factorial de un número, estudiamos la evolución de la pila de llamadas a lo largo de la ejecución del programa utilizando la ventana **Call stack** de CodeBlocks.

8. Segundo ejemplo de aplicación

Vamos a ver un nuevo ejemplo de aplicación de recursividad. Se trata de la búsqueda binaria de un elemento, que en este caso, se encuentra dentro de un arreglo ordenado (también podría darse el caso de estar dentro de una lista ordenada).

Estudiaremos la implementación de este algoritmo de manera recursiva y además haremos una breve comparativa con la búsqueda secuencial de un elemento dentro de un arreglo que también se encuentra ordenado. El objetivo de la comparativa es justificar la elección de un algoritmo frente a otro en función del problema a resolver.

Volviendo a la búsqueda binaria, su lógica se basa en dividir en 2 (es decir, a la mitad) el espacio de búsqueda a lo largo de cada iteración. En la iteración se verifica en cuál de las dos particiones se encuentra contenido el valor buscado, se queda con esa partición y descarta la otra. El proceso se repite hasta encontrar (o no) el valor buscado. En la figura 5 se puede ver que el espacio de búsqueda se va reduciendo en cada iteración, encuadrándose en el enfoque de *divide y vencerás*, lo que lleva a pensar que el algoritmo es candidato a ser resuelto de manera recursiva.



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

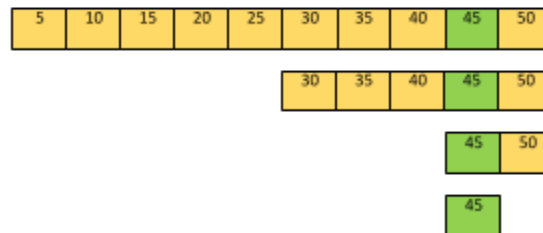


Figura 5. Búsqueda binaria de un elemento dentro de un arreglo ordenado.

En la figura 6 se puede ver la implementación de la función **busquedaBinariaRecursiva**.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int busquedaBinariaRecursiva(const int *arr, int izquierda, int derecha, int x)
5  {
6      if (derecha >= izquierda)
7      {
8          int mitad = izquierda + (derecha - izquierda) / 2;
9
10         /** Si el elemento está presente en el medio */
11         if (arr[mitad] == x)
12             return mitad;
13
14         /** Si el elemento es menor que el mid, entonces puede estar en el
15         sub arreglo izquierdo. */
16         if (arr[mitad] > x)
17             return busquedaBinariaRecursiva(arr, izquierda, mitad - 1, x);
18
19         /** De lo contrario, el elemento solo puede estar en el sub arreglo
20         derecho. */
21         return busquedaBinariaRecursiva(arr, mitad + 1, derecha, x);
22     }
23
24     /** Elemento no encontrado */
25     return -1;
26 }

```

Figura 6. Implementación de la función **busquedaBinariaRecursiva**.

La función recibe como argumentos: el arreglo sobre el que se va a realizar la búsqueda, el subíndice del elemento en dónde inicia la búsqueda dentro del arreglo, el subíndice del elemento en dónde finaliza la búsqueda dentro del arreglo, y finalmente, el elemento buscado.



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

La función retornará el subíndice de la posición que ocupa el elemento dentro del arreglo. En el caso de que el elemento buscado no exista retornará -1.

En la línea 37 de la figura 7 se muestra la función **main** desde la que se invoca a la función **busquedaBinariaRekursiva**. En la invocación inicial se le pasa a la función los subíndices inicial (0) y final (9) del arreglo (si observan podrán ver que posee 10 elementos), esto cubre toda su extensión. Luego, en las sucesivas iteraciones, los subíndices irán variando de manera tal que el espacio de búsqueda se irá reduciendo hasta dar (o no) con la solución.

En la figura 8 se puede ver la pila de llamadas a la función de búsqueda. La columna **Nr** muestra el orden. En la "base" de la pila (Nr = 3) se encuentra **main**, luego las llamadas a **busquedaBinariaRekursiva**. Si se detienen a ver los argumentos (particularmente las variables **izquierda** y **derecha**) verán como efectivamente el espacio de búsqueda se reduce.

La figura 9 muestra la evolución de las variables que contienen los índices cómo evoluciona el espacio de búsqueda de la solución a lo largo de todo el ejemplo.

```
28  int main()
29  {
30      int arreglo[] = {5, 10, 15, 20, 25, 30, 35, 40, 45, 50};
31      int nroElementos;
32      int resultado;
33      int buscado = 45;
34
35      nroElementos = sizeof(arreglo) / sizeof(arreglo[0]);
36      resultado = busquedaBinariaRekursiva(arreglo, 0, nroElementos - 1, buscado);
37      if(resultado != -1)
38      {
39          printf("El valor %d se encuentra en la posición %d del arreglo.\n",
40                buscado,
41                resultado);
42      }
43      else
44      {
45          printf("No se ha encontrado el valor %d en el arreglo.\n", buscado);
46      }
47      return 0;
48  }
```

Figura 7. Invocación de la función *busquedaBinariaRekursiva*.



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

Call stack		
Nr	Address	Function
0		busquedaBinariaRecursiva (arr=0x61fde0, izquierda=8, derecha=9, x=45)
1	0x401603	busquedaBinariaRecursiva(arr=0x61fde0, izquierda=5, derecha=9, x=45)
2	0x401603	busquedaBinariaRecursiva(arr=0x61fde0, izquierda=0, derecha=9, x=45)
3	0x401691	main()

Figura 8. Pila de llamadas a la función *busquedaBinariaRecursiv* desde la función *main()*.

Ahora, unas preguntas:

Pregunta 1: ¿Puedes explicar qué sucede en la línea 35 de la figura 7?

No vale responder solo: "calcular la cantidad de elementos del arreglo". Además de ello, que significan los elementos que entran en juego para el cálculo.

Pregunta 2: Esta pregunta es MUY IMPORTANTE. ¿Puedes identificar la condición de base en la función *busquedaBinariaRecursiva*?



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

Subíndice del arreglo

Arreglo

Elemento buscado

0	1	2	3	4	5	6	7	8	9
5	10	15	20	25	30	35	40	45	50

45

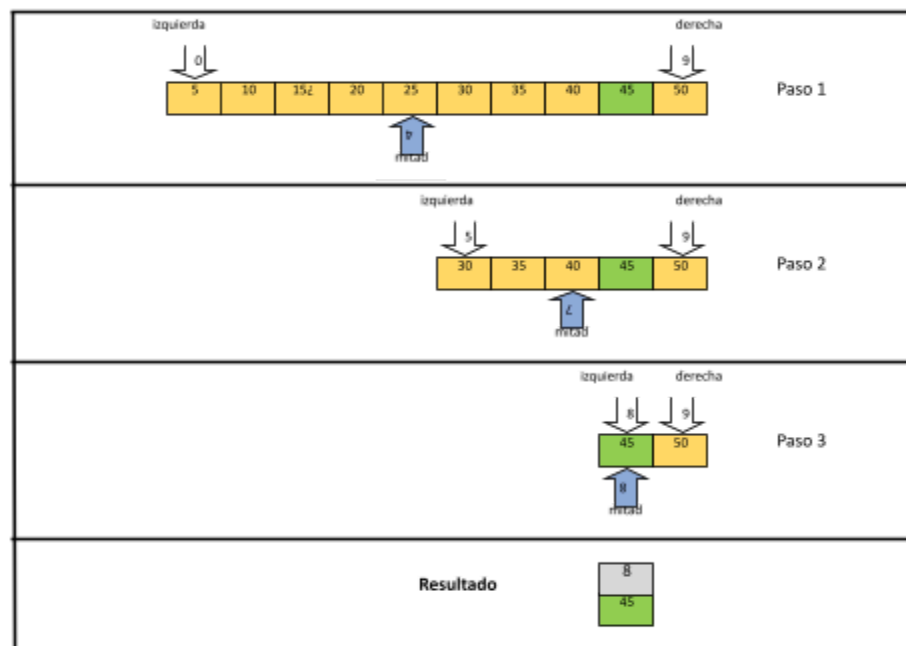


Figura 9. Búsqueda binaria de un elemento dentro de un arreglo ordenado.

Para finalizar, vamos a ver cuál es el comportamiento del algoritmo de búsqueda binaria en relación al tamaño del espacio de búsqueda. Deseo poner de manifiesto que esta última parte es un complemento a los temas que hemos tratado de recursividad, y se centra en la selección del algoritmo en función del tipo de problema a resolver.



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

Ejercicios propuestos

Ejercicio Propuesto 1: Modifica el ejercicio de búsqueda secuencial recursiva para que el espacio de búsqueda sea de 2^{21} elementos de tipo entero. Mide el tiempo de ejecución del algoritmo. Luego repite el experimento utilizando una búsqueda secuencial. Puedes utilizar el ejemplo a continuación para medir el tiempo de ejecución.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  /** Funcion cuyo tiempo de ejecucion se desea medir. */
6  void miFuncion()
7  {
8      long int x;
9      for(x = 0; x <= 1000000000; x++);
10 }
11
12 int main()
13 {
14     clock_t inicio, fin;
15     double tiempo_total;
16
17     /** Marca de tiempo de inicio de ejecucion */
18     inicio = clock();
19
20     /** Llamada a la función que se desea medir */
21     miFuncion();
22
23     /** Marca de tiempo de fin de ejecucion */
24     fin = clock();
25
26     tiempo_total = ((double) (fin - inicio)) / CLOCKS_PER_SEC;
27     printf("Tiempo de ejecución: %f segundos\n", tiempo_total);
28
29     return 0;
30 }
```

**UNLaM**Dto. Ingeniería e Investigaciones Tecnológicas

Ejercicio Propuesto 2: Modifica el ejercicio de búsqueda secuencial recursiva para que pueda realizar búsquedas por DNI en un arreglo que contenga datos del tipo Persona. Diseña el tipo de datos tPersona (además de DNI, incluye el apellido, nombre y fecha de nacimiento). Debes diseñar, implementar y utilizar una función de comparación para datos de tipo persona. La función de búsqueda binaria recursiva debe recibir entre sus parámetros un puntero a la función de comparación. Crea las condiciones de entorno apropiadas para que se pueda ejecutar y probar.

Si has llegado hasta aquí te agradezco por tu atención y aprovecho para ayudarte con la respuesta a la Pregunta 2. En caso de que no hayas encontrado la respuesta, ésta se encuentra en la línea 6 de la figura 6.



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

Complejidad computacional de los algoritmos

(ESTE TEMA SE ESTUDIARÁ EN LA MATERIA “PROGRAMACIÓN AVANZADA” solo se menciona a título informativo en este apunte)

En una **búsqueda binaria**, cuando la cantidad de elementos del espacio de búsqueda es potencia de 2 (por ejemplo, 1, 2, 4, 8, 16, 32, ,etc) la cantidad de iteraciones máxima a realizar para encontrar un elemento dado se encuentra determinada por la expresión:

Número de iteraciones = $\log_2 n$

En donde “**n**” es la cantidad de elementos del arreglo en donde se realizará la búsqueda.

Por otro lado, en el caso de que la cantidad de elementos no sea potencia de 2, el número de iteraciones máximo estará dado por:

Número de iteraciones (peor caso) = $\lceil \log_2 n \rceil + 1$

Para nuestro ejemplo, $n = 10$, por ello la cantidad de iteraciones máxima será:

$(\log_2 10) + 1 = 3,32 + 1 \approx 4$

En contraste, si en lugar de una **búsqueda binaria** hiciéramos una **búsqueda secuencial** sobre el mismo arreglo, la cantidad de iteraciones para encontrar el elemento 45 hubieran sido 9. Es decir, en el peor de los casos, hubiera sido necesario recorrer todo el vector, por ende, el número de iteraciones hubiera sido:

Número de iteraciones (peor caso) = n



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

Veamos la siguiente tabla en que muestra la cantidad de iteraciones máximas según la cantidad de elementos (n) entre los cuales hay que buscar:

n	$\log_2 n$
1	0
2	1
4	2
8	3
16	4
32	5
64	6
128	7
256	8
512	9
1024	10

Cuando la cantidad de elementos del arreglo es 1024 (2^{10}) el número máximo de iteraciones para encontrar el elemento buscado en el peor de los casos sería 10. En cambio, si se hiciera una búsqueda secuencial, en el peor de los casos serían necesarias 1024 iteraciones.

Vemos que la búsqueda binaria es menos costosa que la búsqueda secuencial frente a las mismas condiciones.

Existe una manera de expresar cuánto tiempo tarda un algoritmo en ejecutarse en función del tamaño de la entrada de datos (en este caso, el número de elementos en el arreglo) y se denomina **Big O**.

Para la búsqueda binaria el tiempo de ejecución estará en el orden $O(\log_2 n)$, mientras que en la búsqueda secuencial estará en el orden $O(n)$.

Finalmente, para ambos algoritmos debemos tener en cuenta las siguientes características:

- **Búsqueda Binaria $O(\log_2 n)$:** Es rápida para arreglos grandes ya que el número de comparaciones necesarias crece muy lentamente en comparación con el tamaño del arreglo. Requiere que el arreglo esté ordenado previamente.
- **Búsqueda Secuencial $O(n)$:** Puede ser lenta para arreglos grandes ya que potencialmente necesita revisar cada elemento. Funciona en arreglos ordenados y

**UNLaM**Dto. Ingeniería e Investigaciones Tecnológicas

desordenados, pero es menos eficiente en términos de tiempo en comparación con la búsqueda binaria cuando el arreglo está ordenado.

Lo que podemos observar es que la elección del algoritmo va de la mano con el tipo de problema a resolver.



UNLaM

Dto. Ingeniería e Investigaciones Tecnológicas

Índice de figuras

<i>Figura 1.Descomposición de la función factorial en su caso base y recursivo.</i>	5
<i>Figura 2.Implementación recursiva de la función factorial del un número entero.</i>	7
<i>Figura 3.Implementación recursiva de la función factorial con variables temporales para visualizar los resultados intermedios.</i>	8
<i>Figura 4. Vista en modo Debug de las condiciones iniciales del programa.</i>	9
<i>Figura 5. Búsqueda binaria de un elemento dentro de un arreglo ordenado.</i>	20
<i>Figura 6. Implementación de la función <code>busquedaBinariaRecursiva</code>.</i>	21
<i>Figura 7. Invocación de la función <code>busquedaBinariaRecursiva</code>.</i>	22
<i>Figura 8. Pila de llamadas a la función <code>busquedaBinariaRecursiva</code> desde la función <code>main()</code>.</i>	22
<i>Figura 9. Búsqueda binaria de un elemento dentro de un arreglo ordenado.</i>	23

Bibliografía

- [1] B. W. Kernighan y D. M. Ritchie, El lenguaje de programación C, Pearson Educación, 1991.
- [2] H. M. Deitel y P. J. Deitel, Cómo programar en C/C+, Pearson Educación, 1995.
- [3] Herbert Schildt, Turbo C/C++ 3.1 Manual de referencia, Osborne/McGraw-Hill, 1994.